Reinforcement Learning

Control as Inference, Inverse RL, and RL with LLMs

SJTU

Outline

1. Control as Inference Recap
2. Control as Variational Inference
3. Maximum Entropy RL Algorithms
4. Inverse Reinforcement Learning
5. RL and LLMs: The Basics
6. Algorithms for RL with LLMs
7. Preferences, RLHF, and Verifiers
8. RL with Partial Observability (Bonus)

A Probabilistic Graphical Model of Decision Making

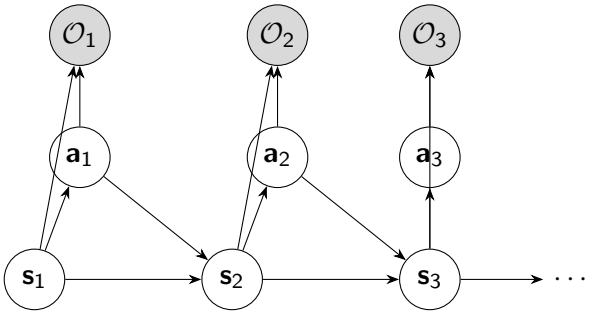
Replace the deterministic optimization:

$$\mathbf{a}_1, \dots, \mathbf{a}_T = \arg\max_{\mathbf{a}_1,\dots,\mathbf{a}_T} \sum_{t=1}^{T} r(\mathbf{s}_t, \mathbf{a}_t)$$

with a **probabilistic** model — no assumption of optimal behavior!

Introduce **optimality variables** $\mathcal{O}_t$:

$$p(\mathcal{O}_t|\mathbf{s}_t, \mathbf{a}_t) = \exp(r(\mathbf{s}_t, \mathbf{a}_t))$$



$$p(\tau|\mathcal{O}_{1:T}) \propto p(\tau) \exp\Big(\sum_t r(\mathbf{s}_t, \mathbf{a}_t)\Big)$$

Policy Computation via Backward Messages

For $t = T - 1$ to 1:

$$Q_t(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + \log \mathbb{E}\big[\exp(V_{t+1}(\mathbf{s}_{t+1}))\big]$$

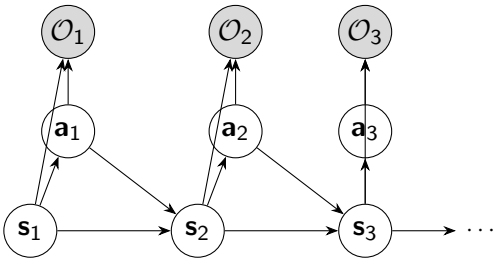
$$V_t(\mathbf{s}_t) = \log \int \exp(Q_t(\mathbf{s}_t, \mathbf{a}_t))\, d\mathbf{a}_t$$

The optimal policy under this model:

$$\pi(\mathbf{a}_t|\mathbf{s}_t) = \exp\big(Q_t(\mathbf{s}_t, \mathbf{a}_t) - V_t(\mathbf{s}_t)\big) = \exp\big(A_t(\mathbf{s}_t, \mathbf{a}_t)\big)$$

Policy is the **exponentiated advantage** — soft version of taking the arg max

Control as Inference: Summary



1. Probabilistic graphical model for optimal control
2. Control = inference (similar to HMM, EKF, etc.)
3. Very similar to dynamic programming / value iteration (but "soft")

The Optimism Problem

Recall backward messages:

$$\beta_t(\mathbf{s}_t, \mathbf{a}_t) = p(\mathcal{O}_t|\mathbf{s}_t, \mathbf{a}_t)\, \mathbb{E}_{\mathbf{s}_{t+1}\sim p(\mathbf{s}_{t+1}|\mathbf{s}_t,\mathbf{a}_t)}\big[\beta_{t+1}(\mathbf{s}_{t+1})\big]$$

Let $V_t(\mathbf{s}_t) = \log \beta_t(\mathbf{s}_t), \quad Q_t(\mathbf{s}_t, \mathbf{a}_t) = \log \beta_t(\mathbf{s}_t, \mathbf{a}_t)$

Then:

$$Q_t(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + \underbrace{\log \mathbb{E}\big[\exp(V_{t+1}(\mathbf{s}_{t+1}))\big]}_{\text{“optimistic” transition!}}$$

The inference problem $p(\mathbf{s}_{1:T}, \mathbf{a}_{1:T}|\mathcal{O}_{1:T})$ yields:

- Policy: $p(\mathbf{a}_t|\mathbf{s}_t, \mathcal{O}_{1:T})$ — we want this
- Transition: $p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t, \mathcal{O}_{1:T}) \neq p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$ — optimistic!

Addressing the Optimism Problem

Question: Can we find $q(\mathbf{s}_{1:T}, \mathbf{a}_{1:T})$ that is close to $p(\mathbf{s}_{1:T}, \mathbf{a}_{1:T}|\mathcal{O}_{1:T})$ but uses the *true* dynamics $p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$?

Let $\mathbf{x} = \mathcal{O}_{1:T}$ and $\mathbf{z} = (\mathbf{s}_{1:T}, \mathbf{a}_{1:T})$

Find $q(\mathbf{z})$ to approximate $p(\mathbf{z}|\mathbf{x})$

This is **variational inference**!

Control via Variational Inference

Define the variational distribution:

$$q(\mathbf{s}_{1:T}, \mathbf{a}_{1:T}) = p(\mathbf{s}_1) \prod_t p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)\, q(\mathbf{a}_t|\mathbf{s}_t)$$

True posterior $p(\mathbf{s}_{1:T}, \mathbf{a}_{1:T}|\mathcal{O}_{1:T})$:

- Has optimality variables $\mathcal{O}_t$
- Transition conditioned on $\mathcal{O}_{1:T}$

$\rightarrow$ "Optimistic" dynamics

Variational approx. $q(\mathbf{s}_{1:T}, \mathbf{a}_{1:T})$:

- Same dynamics $p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$
- Same initial state $p(\mathbf{s}_1)$
- Only new thing: $q(\mathbf{a}_t|\mathbf{s}_t)$

The Variational Lower Bound (ELBO)

Standard ELBO: $\log p(\mathbf{x}) \geq \mathbb{E}_{\mathbf{z}\sim q(\mathbf{z})}[\log p(\mathbf{x}, \mathbf{z}) - \log q(\mathbf{z})]$

Substituting $\mathbf{x} = \mathcal{O}_{1:T}$, $\mathbf{z} = (\mathbf{s}_{1:T}, \mathbf{a}_{1:T})$:

$$\log p(\mathcal{O}_{1:T}) \geq \mathbb{E}_{(\mathbf{s}_{1:T}, \mathbf{a}_{1:T})\sim q}\Big[\sum_{t=1}^{T} \log p(\mathcal{O}_t|\mathbf{s}_t, \mathbf{a}_t) - \sum_{t=1}^{T} \log q(\mathbf{a}_t|\mathbf{s}_t)\Big]$$

$$= \mathbb{E}_{(\mathbf{s}_{1:T}, \mathbf{a}_{1:T})\sim q}\Big[\sum_{t} r(\mathbf{s}_t, \mathbf{a}_t) - \log q(\mathbf{a}_t|\mathbf{s}_t)\Big]$$

$$= \sum_{t} \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t)\sim q}\big[r(\mathbf{s}_t, \mathbf{a}_t) + \mathcal{H}(q(\mathbf{a}_t|\mathbf{s}_t))\big]$$

Maximize reward **and** maximize action entropy!

Backward Pass: Variational Version

For $t = T - 1$ to 1:

$$Q_t(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + \mathbb{E}\big[V_{t+1}(\mathbf{s}_{t+1})\big]$$

$$V_t(\mathbf{s}_t) = \log \int \exp(Q_t(\mathbf{s}_t, \mathbf{a}_t))\, d\mathbf{a}_t$$

Value iteration:

1. $Q(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}[V(\mathbf{s}')]$
2. $V(\mathbf{s}) \leftarrow \max_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a})$

Soft value iteration:

1. $Q(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}[V(\mathbf{s}')]$
2. $V(\mathbf{s}) \leftarrow \text{soft}\max_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a})$

Variants:

- Discounted: $Q_t(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + \gamma \mathbb{E}[V_{t+1}(\mathbf{s}_{t+1})]$
- Temperature: $V_t(\mathbf{s}_t) = \alpha \log \int \exp\big(\frac{1}{\alpha} Q_t(\mathbf{s}_t, \mathbf{a}_t)\big)\, d\mathbf{a}_t$

Soft Q-Learning

Standard Q-learning:

$$\phi \leftarrow \phi + \alpha \nabla_\phi Q_\phi(\mathbf{s}, \mathbf{a}) \big(r(\mathbf{s}, \mathbf{a}) + \gamma V(\mathbf{s}') - Q_\phi(\mathbf{s}, \mathbf{a})\big)$$

Target value: $V(\mathbf{s}') = \max_{\mathbf{a}'} Q_\phi(\mathbf{s}', \mathbf{a}')$

Soft Q-learning:

$$\phi \leftarrow \phi + \alpha \nabla_\phi Q_\phi(\mathbf{s}, \mathbf{a}) \big(r(\mathbf{s}, \mathbf{a}) + \gamma V(\mathbf{s}') - Q_\phi(\mathbf{s}, \mathbf{a})\big)$$

Target value: $V(\mathbf{s}') = \text{soft}\max_{\mathbf{a}'} Q_\phi(\mathbf{s}', \mathbf{a}') = \log \int \exp(Q_\phi(\mathbf{s}', \mathbf{a}'))\, d\mathbf{a}'$

Policy: $\pi(\mathbf{a}|\mathbf{s}) = \exp(Q_\phi(\mathbf{s}, \mathbf{a}) - V(\mathbf{s})) = \exp(A(\mathbf{s}, \mathbf{a}))$

Soft Q-Learning Algorithm

1. Take action $\mathbf{a}_i$ and observe $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)$, add to replay buffer $\mathcal{R}$
2. Sample mini-batch $\{\mathbf{s}_j, \mathbf{a}_j, \mathbf{s}'_j\}$ from $\mathcal{R}$ uniformly
3. Compute $y_j = r(\mathbf{s}_j, \mathbf{a}_j) + \gamma$ soft $\max_{\mathbf{a}'_j} Q_{\bar{\theta}}(\mathbf{s}'_j, \mathbf{a}'_j)$ using *target* network $Q_{\bar{\theta}}$
4. $\theta \leftarrow \theta - \alpha \sum_j \frac{dQ_\theta}{d\theta}(\mathbf{s}_j, \mathbf{a}_j)(Q_\theta(\mathbf{s}_j, \mathbf{a}_j) - y_j)$
5. Update $\bar{\theta}$: copy θ every N steps

Policy Gradient with Soft Optimality

Recall the ELBO:

$$\log p(\mathcal{O}_{1:T}) \geq \sum_t \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t) \sim q}\big[r(\mathbf{s}_t, \mathbf{a}_t) + \mathcal{H}(q(\mathbf{a}_t|\mathbf{s}_t))\big] = \mathcal{L}(q)$$

Policy gradient objective:

$$J(\theta) = \sum_t \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t) \sim \pi_\theta}\big[r(\mathbf{s}_t, \mathbf{a}_t)\big] + \mathbb{E}_{\pi(\mathbf{s}_t)}\big[\mathcal{H}(\pi_\theta(\mathbf{a}_t|\mathbf{s}_t))\big]$$

- Adding an entropy term to policy gradient is very common
- Entropy has a closed-form expression for most common policy classes

Soft Actor-Critic (SAC)

Off-policy actor-critic with soft optimality:

1. Get $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)$ by taking $\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s})$; store in replay buffer
2. Evaluate $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \mathbb{E}_{\mathbf{a}' \sim \pi_\theta(\mathbf{a}'|\mathbf{s}'_i)}\big[\hat{Q}^\pi_\phi(\mathbf{s}'_i, \mathbf{a}') - \log \pi_\theta(\mathbf{a}'|\mathbf{s}'_i)\big]$
3. Update ϕ using $\nabla_\phi \sum_{i=1}^{B} \| \hat{Q}^\pi_\phi(\mathbf{s}_i, \mathbf{a}_i) - y_i \|^2$
4. $J(\theta) = \sum_i \mathbb{E}_{\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s}_i)}\big[\hat{Q}^\pi_\phi(\mathbf{s}_i, \mathbf{a}) + \mathcal{H}(\pi_\theta(\mathbf{a}|\mathbf{s}_i))\big]$
5. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

One of the most commonly used off-policy actor-critic algorithms!

Usually add a **temperature** β in front of $\mathcal{H}(\pi_\theta(\mathbf{a}|\mathbf{s}_i))$

Why Learn Reward Functions?

The imitation learning perspective:

Standard imitation learning:

- Copy the *actions* of the expert
- No reasoning about outcomes

Human imitation learning:

- Copy the *intent* of the expert
- Might take very different actions!

The RL perspective:

- Some environments have obvious rewards (e.g., game score)
- Many real-world tasks: what is the reward? (e.g., autonomous driving)

Inverse Reinforcement Learning

Goal: Infer **reward functions** from **demonstrations**

- By itself, this is an **underspecified** problem
- Many reward functions can explain the **same** behavior

Key idea from control as inference:
$p(\mathcal{O}_t|\mathbf{s}_t, \mathbf{a}_t) \propto \exp(r_\psi(\mathbf{s}_t, \mathbf{a}_t))$ with learnable parameters ψ

Given: samples $\{\tau_i\}$ from expert $\pi^\star(\tau)$

$$p(\tau|\mathcal{O}_{1:T}, \psi) \propto p(\tau) \exp\Big(\sum_t r_\psi(\mathbf{s}_t, \mathbf{a}_t)\Big)$$

The Boltzmann Trajectory Distribution

In MaxEnt IRL, we assume a trajectory τ follows an **exponential (Boltzmann) distribution** given the reward parameters ψ:

$$p(\tau \mid \mathcal{O}_{1:T}, \psi) = \frac{p(\tau)\, \exp\big(r_\psi(\tau)\big)}{Z(\psi)}$$

where:

- $r_\psi(\tau) = \sum_{t=1}^{T} r_\psi(\mathbf{s}_t, \mathbf{a}_t)$ — cumulative trajectory reward
- $p(\tau) = p(\mathbf{s}_1) \prod_t p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$ — prior dynamics (initial-state distribution and transitions)
- $Z(\psi) = \int p(\tau) \exp(r_\psi(\tau))\, d\tau$ — **partition function** (normalizing constant)

Higher-reward trajectories are exponentially more likely, weighted by feasibility $p(\tau)$.

Maximum Likelihood IRL

Maximum likelihood learning:

$$\max_{\psi} \frac{1}{N} \sum_{i=1}^{N} \log p(\tau_i | \mathcal{O}_{1:T}, \psi) = \max_{\psi} \frac{1}{N} \sum_{i=1}^{N} r_\psi(\tau_i) - \log Z$$

where $Z = \int p(\tau) \exp(r_\psi(\tau))\, d\tau$ is the **partition function** (the hard part!)

Gradient:

$$\nabla_\psi \mathcal{L} = \underbrace{\mathbb{E}_{\tau \sim \pi^\star(\tau)}[\nabla_\psi r_\psi(\tau_i)]}_{\text{expert samples}} - \underbrace{\mathbb{E}_{\tau \sim p(\tau | \mathcal{O}_{1:T}, \psi)}[\nabla_\psi r_\psi(\tau)]}_{\text{soft optimal policy under current reward}}$$

Estimating the IRL Gradient

Idea: learn $p(\mathbf{a}_t|\mathbf{s}_t, \mathcal{O}_{1:T}, \psi)$ using any max-ent RL algorithm, then sample $\{\tau_j\}$

$$\nabla_\psi \mathcal{L} \approx \frac{1}{N}\sum_{i=1}^{N} \nabla_\psi r_\psi(\tau_i) - \frac{1}{M}\sum_{j=1}^{M} \nabla_\psi r_\psi(\tau_j)$$

Problem: Running RL to convergence each step is expensive!

Solution 1: "Lazy" policy optimization + importance sampling

$$\nabla_\psi \mathcal{L} \approx \frac{1}{N}\sum_{i=1}^{N} \nabla_\psi r_\psi(\tau_i) - \frac{1}{\sum_j w_j}\sum_{j=1}^{M} w_j \nabla_\psi r_\psi(\tau_j)$$

where $w_j = \frac{\exp(\sum_t r_\psi(\mathbf{s}_t, \mathbf{a}_t))}{\prod_t \pi(\mathbf{a}_t|\mathbf{s}_t)}$

IRL Looks Like a Game...

Policy (generator): $\pi_\theta(\tau)$

$\downarrow$ samples from $\pi_\theta(\tau)$

$\nabla_\theta \mathcal{L} \approx \frac{1}{M} \sum_{j=1}^{M} \nabla_\theta \log \pi_\theta(\tau_j) r_\psi(\tau_j)$

Policy changed to make it *harder* to distinguish from demos

Demonstrations: $\pi^\star(\tau)$

$\downarrow$ samples from $\pi^\star(\tau)$

$\nabla_\psi \mathcal{L} \approx$

$\frac{1}{N} \sum_i \nabla_\psi r_\psi(\tau_i) - \frac{1}{\sum_j w_j} \sum_j w_j \nabla_\psi r_\psi(\tau_j)$

Demos made more likely, samples less likely

This adversarial structure resembles GANs!

Generative Adversarial Networks (GANs)

Generator: $p_\theta(\mathbf{x}|\mathbf{z})$

$\theta \leftarrow \arg\max_\theta \mathbb{E}_{\mathbf{x}\sim p_\theta}\big[\log D_\psi(\mathbf{x})\big]$

Discriminator: $D(\mathbf{x}) = p_\psi(\text{real}|\mathbf{x})$

$\psi = \arg\max_\psi \frac{1}{N}\sum_{\mathbf{x}\sim p^\star} \log D_\psi(\mathbf{x}) + \frac{1}{M}\sum_{\mathbf{x}\sim p_\theta} \log(1 - D_\psi(\mathbf{x}))$

Optimal discriminator:

$$D^\star(\mathbf{x}) = \frac{p^\star(\mathbf{x})}{p_\theta(\mathbf{x}) + p^\star(\mathbf{x})}$$

Goodfellow et al. '14

Inverse RL as a GAN

For IRL, optimal policy approaches $\pi_\theta(\tau) \propto p(\tau)\exp(r_\psi(\tau))$

Choose discriminator parameterization:

$$D_\psi(\tau) = \frac{\frac{1}{Z}\exp(r(\tau))}{\prod_t \pi_\theta(\mathbf{a}_t|\mathbf{s}_t) + \frac{1}{Z}\exp(r(\tau))}$$

Optimize:

$$\psi \leftarrow \arg\max_\psi \mathbb{E}_{\tau\sim p^\star}\big[\log D_\psi(\tau)\big] + \mathbb{E}_{\tau\sim\pi_\theta}\big[\log(1 - D_\psi(\tau))\big]$$

Importance weights are subsumed into Z!

Finn*, Christiano* et al. "A Connection Between GANs, IRL, and Energy-Based Models."

Can We Just Use a Regular Discriminator? (GAIL)

Generator/policy $\pi_\theta(\tau)$:
$\nabla_\theta \mathcal{L} \approx \frac{1}{M} \sum_{j=1}^{M} \nabla_\theta \log \pi_\theta(\tau_j) \log D_\psi(\tau_j)$

$D_\psi(\tau) =$ standard binary classifier

Data/demonstrations $p^\star(\tau)$:
$\psi \leftarrow \arg\max_\psi \mathbb{E}_{\tau \sim p^\star}\big[\log D_\psi(\tau)\big] + \mathbb{E}_{\tau \sim \pi_\theta}\big[\log(1 - D_\psi(\tau))\big]$

Pros & cons:

+ Often simpler, fewer moving parts
– Discriminator knows *nothing* at convergence
– Generally cannot reoptimize the "reward"

Ho & Ermon. Generative adversarial imitation learning.

IRL as Adversarial Optimization: Summary

MaxEnt IRL:

- Learns distribution $p(\tau)$ s.t. demos have max likelihood
- $p(\tau) \propto \exp(r(\tau))$
- Reward function is the output

$$D(\tau) = \frac{\frac{1}{Z}\exp(r(\tau))}{\frac{1}{Z}\exp(r(\tau)) + \pi(\tau)}$$

GAIL:

- $D(\tau)$ = probability τ is a demo
- Use $\log D(\tau)$ as "reward"
- Classifier is the output

$D(\tau)$ = some binary classifier

Actually the same thing!

Language Models

Autoregressive transformer:

- Input: tokens $x_0, x_1, \ldots, x_n$
- Output: next-token predictions $\hat{x}_1, \hat{x}_2, \ldots, \hat{x}_{n+1}$
- Masked self-attention $\rightarrow$ position-wise MLP $\rightarrow$ softmax

- Language models are often trained with **supervised learning**
- But we can also train them with **RL** to maximize some reward function
- Key questions:
  - What is the (PO)MDP?
  - What is the reward?
  - What algorithm to use?

The Anatomy of LLM Training

Pre-training:

- Scrape large amounts of diverse data (Common Crawl, Wikipedia, etc.)
- Train with next-token prediction
- Gives the LLM *knowledge*

Post-training:

- High-quality SFT datasets
- RL finetuning (e.g., RLHF)
- Tells the LLM how to *use* this knowledge

LLM Post-Training

Pre-trained LLMs are **not** agents or assistants — they are "text completion" engines
. . . but completing *any* text requires significant "world knowledge"

Basic post-training: instruction tuning

- Fine-tune on (instruction, response) pairs
- Multi-task instruction finetuning (1.8K+ tasks)
- Enables generalization to unseen tasks

Challenges:

- Typically very labor-intensive
- Especially for skilled tasks (e.g., coding, math)

Post-Training with RL from Preferences

Three-step process (RLHF):

1. **Collect demonstration data**, train a supervised policy (SFT)
2. **Collect comparison data**, train a reward model
   - Sample multiple model outputs
   - Human labeler ranks outputs from best to worst
   - Train reward model on preferences
3. **Optimize policy** against the reward model using RL (e.g., PPO)

Post-Training with RL from Verifiers

Use rewards indicative of answer **correctness**

Train model to generate "thoughts" that lead to more correct answers

Large Reasoning Models (e.g., DeepSeek-R1):

- Extended thinking process
- Self-verification and backtracking
- "Wait, let me check that..."

Standard LLM: single chain of thought

Reasoning LLM: extended deliberation with self-correction

Key: the **thinking process** is trained via RL, not supervised learning

A Basic RL Formulation for LLMs

One-step formulation:

Context/prompt/prefix = state $\mathbf{s}$
Completion = action $\mathbf{a}$

$$p(\mathbf{a}|\mathbf{s}) = p(x_5|x_{1:4})\, p(x_6|x_{1:4}, x_5)$$

$\pi_\theta(\mathbf{a}|\mathbf{s})$: the LLM policy

Objective: $\mathbb{E}_{\pi_\theta(\mathbf{a}|\mathbf{s})}[r(\mathbf{s}, \mathbf{a})]$

Multi-step (token-level) formulation:

$\pi_\theta(\mathbf{a}_1|\mathbf{s}_1)\ \pi_\theta(\mathbf{a}_2|\mathbf{s}_2)$

Each token is an action, growing context is the state

Objective: $\mathbb{E}_{\pi_\theta(\tau)}[r(\tau)]$

Multi-step perspective is better when using intermediate rewards and value function baselines

Language Models and Policy Gradients

Policy gradient for the completion:

$$\nabla_\theta \log \pi_\theta(\mathbf{a}|\mathbf{s}) = \nabla_\theta \log p(x_5|x_{1:4}) + \nabla_\theta \log p(x_6|x_{1:4}, x_5)$$

REINFORCE-style estimator:

$$\nabla_\theta \mathbb{E}_{\pi_\theta(\mathbf{a}|\mathbf{s})}[r(\mathbf{s}, \mathbf{a})] \approx \frac{1}{N} \sum_i \nabla_\theta \log \pi_\theta(\mathbf{a}_i|\mathbf{s}_i)\, r(\mathbf{s}_i, \mathbf{a}_i)$$

Importance-weighted estimator (e.g., PPO):

$$\approx \frac{1}{N} \sum_i \frac{\pi_\theta(\mathbf{a}_i|\mathbf{s}_i)}{\bar{\pi}(\mathbf{a}_i|\mathbf{s}_i)} \nabla_\theta \log \pi_\theta(\mathbf{a}_i|\mathbf{s}_i)\, r(\mathbf{s}_i, \mathbf{a}_i)$$

PPO-Style Training Loop for LLMs

$$\nabla_\theta \mathbb{E}_{\pi_\theta(\mathbf{a}|\mathbf{s})}[r(\mathbf{s},\mathbf{a})] \approx \frac{1}{N}\sum_i \frac{\pi_\theta(\mathbf{a}_i|\mathbf{s}_i)}{\bar{\pi}(\mathbf{a}_i|\mathbf{s}_i)} \nabla_\theta \log \pi_\theta(\mathbf{a}_i|\mathbf{s}_i)\, r(\mathbf{s}_i,\mathbf{a}_i)$$

1. Sample batch $\mathcal{B} = \{\mathbf{a}_i\}$, $\mathbf{a}_i \sim \pi_\theta(\mathbf{a}|\mathbf{s})$
2. Evaluate $r(\mathbf{s},\mathbf{a}_i)$ for each $\mathbf{a}_i \in \mathcal{B}$
3. $\bar{\pi} \leftarrow \pi_\theta$
4. Sample minibatch $\mathcal{M} \subset \mathcal{B}$
5. $\theta \leftarrow \theta + \alpha\hat{\nabla}(\theta, \bar{\pi}, \mathcal{M})$

Repeat steps 4–5 K times

Three design choices:

(1) Smartly choose a **baseline**

(2) Smartly choose a **regularizer**

(3) Smartly **set or learn** the reward

Value Function Baselines & GAE

Most often used with GAE (Generalized Advantage Estimation):

$$\delta_{t'} = r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) + \gamma \hat{V}^{\pi}_{\phi}(\mathbf{s}_{t'+1}) - \hat{V}^{\pi}_{\phi}(\mathbf{s}_{t'})$$

$$\hat{A}^{\pi}_{\text{GAE}}(\mathbf{s}_t, \mathbf{a}_t) = \sum_{t'=t}^{\infty} (\gamma\lambda)^{t'-t} \delta_{t'}$$

Value function loss:

$$\mathcal{L}(\phi) = \frac{1}{2} \sum_{i=1}^{N} \left\| \hat{V}^{\pi}_{\phi}(\mathbf{s}_i) - (\hat{A}^{\pi}_{\text{GAE}}(\mathbf{s}_i, \mathbf{a}_i) + \hat{V}_{\text{old}}(\mathbf{s}_i)) \right\|^2$$

Practical notes:

- Either add a new head or make a copy of the entire network
- Don't train the value function on the prompt, only the suffix

Baselining Without Value Functions: GRPO

For each prompt $\mathbf{s}_i$:

- Sample $\mathbf{a}_{i,1}, \ldots, \mathbf{a}_{i,K}$ from $\pi_\theta(\mathbf{a}|\mathbf{s}_i)$
- Set $\hat{A}^\pi_{i,j} = r(\mathbf{s}_i, \mathbf{a}_j) - \frac{1}{K}\sum_{k=1}^{K} r(\mathbf{s}_i, \mathbf{a}_k)$

$$\nabla_\theta \mathbb{E}_{\pi_\theta(\mathbf{a}|\mathbf{s})}[r(\mathbf{s}, \mathbf{a})] \approx \frac{1}{N} \sum_{ij} \frac{\pi_\theta(\mathbf{a}_j|\mathbf{s}_i)}{\bar{\pi}(\mathbf{a}_j|\mathbf{s}_i)} \nabla_\theta \log \pi_\theta(\mathbf{a}_j|\mathbf{s}_i)\, \hat{A}^\pi_{i,j}$$

This is called **GRPO** (Group Relative Policy Optimization)

No value function needed — just compare multiple rollouts per prompt!

Reference Model Regularization

Often we don't want $\pi_\theta(\mathbf{a}|\mathbf{s})$ to drift from reference model $\pi_{\text{ref}}(\mathbf{a}|\mathbf{s})$

Add policy constraint term: $-\mathbb{E}_{\pi_\theta}\big[D_{\text{KL}}(\pi_\theta(\mathbf{a}_t|\mathbf{s}_t),\, \pi_{\text{ref}}(\mathbf{a}_t|\mathbf{s}_t))\big]$

Modified reward:

$$\bar{r}(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) - \beta D_{\text{KL}}\big(\pi_\theta(\mathbf{a}_t|\mathbf{s}_t),\, \pi_{\text{ref}}(\mathbf{a}_t|\mathbf{s}_t)\big)$$

Why?

- Avoid producing "nonsense text" (must be valid language)
- Avoids "exploiting" imperfect reward models

Putting It All Together: PPO for LLMs with GAE

1. Sample $\{\tau^{(i)}\}$ from π_θ *for each prompt*
2. Evaluate $\delta_{t,i} = \bar{r}(\mathbf{s}_{t,i}, \mathbf{a}_{t,i}) + \gamma \hat{V}_\phi^\pi(\mathbf{s}_{t+1,i}) - \hat{V}_\phi^\pi(\mathbf{s}_{t,i})$
3. Evaluate $\hat{A}^\pi_{\text{GAE}}(\mathbf{s}_{t,i}, \mathbf{a}_t(i)) = \sum_{t'=t}^{\infty} (\gamma\lambda)^{t'-t} \delta_{t',i}$
4. Fit $\hat{V}_\phi^\pi(\mathbf{s}_i)$ to targets $\{\hat{A}^\pi_{\text{GAE}}(\mathbf{s}_i, \mathbf{a}_i) + \hat{V}_\phi^\pi(\mathbf{s}_i)\}$
5. $\theta' \leftarrow \theta$
6. $\nabla_{\theta'} J(\theta') \approx \nabla_{\theta'} \mathcal{L}_{\text{CLIP}}(\theta')$
7. $\theta' \leftarrow \theta' + \alpha \nabla_{\theta'} J(\theta')$ (repeat K times)
8. $\theta \leftarrow \theta'$

where $\bar{r}(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) - \beta D_{\text{KL}}(\pi_\theta(\mathbf{a}_t|\mathbf{s}_t), \pi_{\text{ref}}(\mathbf{a}_t|\mathbf{s}_t))$

$$\mathcal{L}_{\text{CLIP}}(\theta') = \sum_{i,t} \min\left\{ \text{clip}\left(\frac{\pi_{\theta'}(\mathbf{a}_t^{(i)}|\mathbf{s}_t^{(i)})}{\pi_\theta(\mathbf{a}_t^{(i)}|\mathbf{s}_t^{(i)})}, 1-\epsilon, 1+\epsilon \right) \hat{A}_t^{(i)}, \ \frac{\pi_{\theta'}(\mathbf{a}_t^{(i)}|\mathbf{s}_t^{(i)})}{\pi_\theta(\mathbf{a}_t^{(i)}|\mathbf{s}_t^{(i)})} \hat{A}_t^{(i)} \right\}$$

Rewards from Human Preferences

RL from human feedback:

1. Sample 2 (or more) trajectories $\tau_i \sim \pi_\theta(\tau)$
2. Ask a person to specify preferences $\{\tau_i \succ \tau_j\}_{i,j}$
3. Train reward $\psi \leftarrow \arg\max_\psi \sum_{i,j} \log \sigma(r_\psi(\tau_i) - r_\psi(\tau_j))$
4. Train policy π_θ with reward r_ψ

Bradley-Terry Model

How do we train r_ψ? Need a probabilistic model!

$$p(\tau_i \succ \tau_j) = \frac{\exp\left(\sum_t r_\psi(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)})\right)}{\exp\left(\sum_t r_\psi(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)})\right) + \exp\left(\sum_t r_\psi(\mathbf{s}_t^{(j)}, \mathbf{a}_t^{(j)})\right)}$$

$$\log p(\tau_i \succ \tau_j) = \log \sigma\big(r_\psi(\tau_i) - r_\psi(\tau_j)\big) \qquad \text{(sigmoid function)}$$

$$\psi \leftarrow \arg\max_\psi \sum_{i,j} \log \sigma\big(r_\psi(\tau_i) - r_\psi(\tau_j)\big)$$

- "Better trajectories are exponentially more likely to be chosen"
- Same method as Elo scores in chess
- Looks a lot like logistic regression

RLHF for LLMs

RL from human feedback:

1. Sample 2+ trajectories $\tau_i \sim \pi_\theta(\tau)$
2. Ask a person to specify preferences
3. Train reward via Bradley-Terry
4. Train policy π_θ with reward r_ψ

Key design choices for LLMs:

- Collect preferences over multiple *on-policy* responses for the *same* prompt
- Reward is assigned only at the *last step* of the generation
- Typically use only 1 or a few iterations (reward model becomes stale!)

Other Reward Sources

Verifier reward:

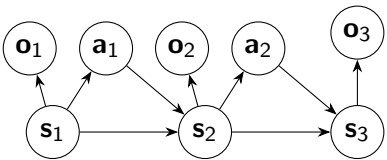
- Model evaluates the **answer**
- Indicates if it is correct or not
- Binary signal at the end

Process reward:

- Model evaluates **each step** of chain-of-thought generation
- Indicates if each step is correct
- Dense reward signal

Both can be used in the same PPO/GRPO training loop!

Beyond MDPs: POMDPs



- Doesn't obey the Markov property
- True state $\mathbf{s}_t$ is not known
- Observations $\mathbf{o}_t$ are partial views

Most real-world problems are POMDPs!

- Autonomous driving
- Robotics
- Video games
- Dialogue systems
- LLM agents

POMDPs Can Be Weird

Example 1: Information-gathering actions

- Must explore to reduce uncertainty about state

Example 2: Stochastic optimal policies

- Unlike MDPs, optimal POMDP policies can be stochastic
- E.g., if reward is equally likely left or right, randomizing is optimal

Which Methods Handle Partial Observability?

Policy gradients: Replace **s** with **o** in policy and advantage

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{H} \nabla_\theta \log \pi_\theta(\mathbf{a}_{i,t}|\mathbf{o}_{i,t})\, A(\mathbf{o}_{i,t}, \mathbf{a}_{i,t})$$

- Policy $\pi_\theta(\mathbf{a}|\mathbf{o})$: OK (no Markov property needed!)
- Advantage $A(\mathbf{o}_t, \mathbf{a}_t)$: takes some care

Key point: advantage is a function of state $\mathbf{s}_t$; it is *not* OK to train $\hat{V}(\mathbf{o}_t)$

Value-based methods: $Q(\mathbf{o}, \mathbf{a}) \leftarrow r(\mathbf{o}, \mathbf{a}) + \gamma \max_{\mathbf{a}'} Q(\mathbf{o}', \mathbf{a}')$

- Do **not** work without the Markov property!

State Space Models for POMDPs

Can we learn a Markovian state space?

$$p(\mathbf{z}) = p(\mathbf{z}_1) \prod_t p(\mathbf{z}_{t+1}|\mathbf{z}_t, \mathbf{a}_t)$$

$$p_\theta(\mathbf{o}|\mathbf{z}) = \prod_t p(\mathbf{o}_t|\mathbf{z}_t)$$

$$q_\phi(\mathbf{z}|\mathbf{o}) = \prod_t q_\phi(\mathbf{z}_t|\mathbf{o}_{1:t})$$

This can work quite well:

$$Q(\mathbf{z}, \mathbf{a}) \leftarrow r(\mathbf{z}, \mathbf{a}) + \gamma \max_{\mathbf{a}'} Q(\mathbf{z}', \mathbf{a}') \quad \checkmark$$

History states (alternative):
$\mathbf{s}_t = \mathbf{o}_{1:t} = (\mathbf{o}_1, \ldots, \mathbf{o}_t)$
Use a **sequence model** (RNN, LSTM, Transformer) to read observation histories

$$Q(\mathbf{o}_{1:t}, \mathbf{a}) \leftarrow r(\mathbf{o}, \mathbf{a}) + \gamma \max_{\mathbf{a}'} Q(\mathbf{o}_{1:t+1}, \mathbf{a}')$$

Partial Observability: Summary

- POMDPs are *weird*: stochastic optimal policies, information-gathering
- Some methods "just do it"
  - But most efficient ones don't, because they require value functions
  - Even those that do only get the best *memoryless* policy
- We could *learn* a Markovian state space with models
- We could also just use *history states*, which means using a sequence model to read in observation histories

Transformers for RL naturally handle partial observability by processing full observation histories!

Summary

1. **Control as Variational Inference:**
   - PGM for decision making with optimality variables
   - Variational inference avoids optimistic transitions
   - Leads to maximum entropy RL objective: $\sum_t \mathbb{E}[r(\mathbf{s}_t, \mathbf{a}_t) + \mathcal{H}(\pi)]$
2. **Max-Ent RL Algorithms:** Soft Q-learning, Soft Actor-Critic (SAC)
3. **Inverse RL:**
   - MaxEnt IRL as maximum likelihood
   - Connection to GANs and adversarial imitation learning (GAIL)
4. **RL with LLMs:**
   - LLM as policy, prompt as state, completion as action
   - PPO/GRPO with baselines, KL regularization, learned rewards
   - RLHF: Bradley-Terry model + policy optimization
   - Verifier rewards for reasoning
5. **Partial Observability:** POMDPs, state space models, history states

Reference: Levine (2018). RL and Control as Probabilistic Inference: Tutorial and Review.